

Алгоритм Хорспула

Пошук підрядка (string matching) є одним із найпоширеніших типів задач через стрімке зростання обсягів текстової інформації. Суть алгоритму полягає у знаходженні входження послідовності символів, відомої як шаблон (pattern), у більшому масиві даних, що називається текстом. Хоча підхід «грубої сили» до цієї проблеми є інтуїтивно зрозумілим і простим у реалізації, він часто виявляється неефективним для великих застосунків, наприклад таких як редагування текстів у реальному часі. Алгоритм Хорспула, запропонований Найджелом Хорспулом у 1980 році як спрощення алгоритму Боєра-Мура, вирішує ці проблеми шляхом використання принципу просторово-часового компромісу.

Постановка задачі

Задача пошуку підрядка формально визначається як пошук першого (або всіх) входжень шаблону $P = P[0..m-1]$ довжиною m символів у тексті $T = T[0..n-1]$ довжиною n символів, де $m \leq n$. Алфавіт Σ складається з усіх символів, які можуть з'явитися в рядках, наприклад, стандартний набір ASCII. Основна мета - визначити індекс i у тексті, де починається шаблон, так що $T[i..i+m-1] = P[0..m-1]$, або повернути значення -1, якщо збіг відсутній.

Найпростішим рішенням був би алгоритм «грубої сили» (наївний алгоритм). Цей метод передбачає вирівнювання шаблону на початку тексту та послідовне порівняння пар символів зліва направо. У разі невідповідності шаблон зміщується рівно на одну позицію вправо, і процес порівняння починається заново з нового вирівнювання. Це триває доти, доки не буде знайдено збіг або поки шаблон не вийде за межі останнього символу тексту.

Характеристика	Алгоритм грубої сили	Алгоритм Хорспула
Напрямок порівняння	Зазвичай зліва направо	Справа наліво
Механізм зсуву	Фіксований на 1 позицію	Змінний на основі таблиці
Попередня обробка	Відсутня	Обов'язкова ($O(m + \sigma)$)
Найкращий випадок	$\Omega(n)$	$\Omega(n/m)$
Найгірший випадок	$O(nm)$	$O(nm)$

Як видно з порівняльної таблиці, метод грубої сили не потребує попередньої обробки, але страждає від відсутності логіки в механізмі зсуву. У найгіршому випадку, наприклад при пошуку шаблону «AAAAAB» у тексті, що складається з тисяч літер «А», наївний алгоритм буде виконувати m порівнянь майже для кожної позиції тексту, що призводить

до складності $O(nm)$. Алгоритм Хорспула покращує цей показник завдяки попередньому аналізу шаблону.

Обґрунтування структур даних: таблиця зсувів

Таблиця зсувів визначає на скільки символів можна безпечно змістити шаблон, щоб не витратити час на вже відомо невдалі порівняння.

Вибір структури для зберігання:

- **Масив:** найкращий варіант для стандартного ASCII (256 символів). Забезпечує миттєвий доступ до значення зсуву за $O(1)$.
- **Словник (хеш-таблиця):** оптимальний для Unicode або розріджених алфавітів. Економить пам'ять, зберігаючи дані лише про символи, що реально є в шаблоні. Для всіх інших символів використовується стандартне значення m .

Логіка розрахунку зсувів:

Аналізується символ тексту, який вирівняний з останнім символом шаблону. Значення зсуву в таблиці визначається так:

1. **Символу немає в шаблоні:** Зсуваємо шаблон на всю його довжину (m).
2. **Символ є в шаблоні:** Зсуваємо так, щоб поєднати символ у тексті з його найправішим входженням у шаблоні (але не в останній позиції). Відстань $= m - 1 - \text{індекс символу}$.

Важливо зазначити, що останній символ шаблону при заповненні таблиці ми не враховуємо. Це гарантує, що зсув завжди буде мінімум на 1 символ. Без цього правила алгоритм міг би отримати зсув 0 і потрапити в нескінченний цикл.

Механізм роботи алгоритму Хорспула

Алгоритм Хорспула складається з двох фаз: попередньої обробки та пошуку.

Фаза попередньої обробки

Під час попередньої обробки алгоритм будує таблицю зсувів, на основі шаблону P та алфавіту Σ . Алгоритм ініціалізує всі записи в таблиці значенням довжини шаблону m . Потім він сканує шаблон зліва направо, охоплюючи індекси від $j = 0$ до $m-2$. Для кожного символу $P[j]$ він перезаписує відповідний запис у таблиці значенням $m - 1 - j$. Оскільки сканування триває зліва направо, остаточне значення, збережене для будь-якого символу, що зустрічається в шаблоні кілька разів, відповідатиме його крайньому правому входженню серед перших $m-1$ символів.

Для реалізації через масив ASCII складність становить $O(m + \sigma)$, де σ - розмір алфавіту. У випадку використання словника складність побудови таблиці наближається до $O(m)$. Термін m відповідає за сканування шаблону, тоді як термін σ представляє час, необхідний для ініціалізації таблиці. У практичному плані для алфавіту ASCII ця фаза виконується майже миттєво навіть для довгих шаблонів.

Фаза пошуку

Фаза пошуку починається з вирівнювання шаблону з початком тексту. На відміну від методу грубої сили, порівняння символів виконується справа наліво, починаючи з останнього символу шаблону. Такий спосіб порівняння дозволяє виконувати більші зсуви шаблону: якщо останній символ шаблону не збігається з текстом, ми можемо потенційно пропустити багато символів одразу на основі таблиці зсувів.

При порівнянні шаблону $P[0..m-1]$ з вікном тексту $T[i-m+1..i]$ (де i - індекс символу тексту, вирівняного з останнім символом шаблону), алгоритм виконує наступні кроки:

1. Порівняти $P[m-1]$ з $T[i]$.
2. Якщо вони збігаються, продовжувати порівняння $P[m-2]$ з $T[i-1]$, $P[m-3]$ з $T[i-2]$ і так далі, рухаючись до початку шаблону.
3. Якщо всі m символів співпадають, входження знайдено, і повертається початковий індекс ($i - m + 1$).
4. Якщо невідповідність виникає в будь-якій точці під час порівняння справа наліво (навіть на першому символі), алгоритм шукає значення зсуву для символу $T[i]$ у таблиці зсувів.
5. Шаблон зсувається вправо на кількість позицій, вказану в таблиці, і процес повторюється.

Слід зазначити, що алгоритм Хорспула для визначення зсуву завжди використовує символ у тексті, вирівняний за *останнім* символом шаблону, незалежно від того, де саме виникла невідповідність.

Але все легше бачити на прикладі, тому..

Покроковий розбір прикладу

Нехай маємо:

- **Текст (Т):** WEATHER_IS_SUMMER_TIME (23 символа)
- **Шаблон (Р):** SUMMER ($m = 6$)

Крок 1. Таблиця зсувів

Дивимось на частину SUMME (без останнього символу).

- **Е:** індекс 4. Зсув: $6 - 1 - 4 = 1$.
- **М:** індекси 2 та 3. Беремо найправіший (3). Зсув: $6 - 1 - 3 = 2$.
- **U:** індекс 1. Зсув: $6 - 1 - 1 = 4$.
- **S:** індекс 0. Зсув: $6 - 1 - 0 = 5$.
- **Усі інші** (включаючи R, пробіл, I, W): зсув дорівнює **6**.

Крок 2. Процес пошуку

Спроба 1:

```
Текст:      W E A T H E R _ I S _ S U M M E R _ T I M E
Шаблон:      S U M M E R
Индекси:     0 1 2 3 4 5
```

1. Порівнюємо останній символ шаблону R з символом тексту під ним: R навпроти E. **Невідповідність.**
2. Дивимось на символ тексту E. У нашій таблиці він дає зсув 1.
3. Зсуваємо шаблон на 1 позицію.

Спроба 2:

Текст: W E A T H E R _ I S _ S U M M E R _ T I M E
 Шаблон: S U M M E R
 Індеси: 1 2 3 4 5 6

1. Порівнюємо з кінця: R (шаблон) == R (текст) - **збіг.**
2. Рухаємось вліво: E = E – **збіг.**
3. Наступний: M і N – **невідповідність.**

(Ми все одно беремо зсув для символу тексту, що стоїть під останньою літерою шаблону. Це літера R на індексі 6.)

4. Для R у таблиці зсув 6.
5. Зсуваємо шаблон на 6 позицій.

Спроба 3:

Текст: W E A T H E R _ I S _ S U M M E R _ T I M E
 Шаблон: S U M M E R
 Індеси: 7 8 9 10 11 12

Тепер кінець шаблону (індекс 12) стоїть над літерою U.

1. Порівнюємо: R (шаблон) і U (текст). **Невідповідність.**
2. Символ U у таблиці дає зсув 4.
3. Зсуваємо шаблон на 4 позиції.

Спроба 4:

Текст: W E A T H E R _ I S _ S U M M E R _ T I M E
 Шаблон: S U M M E R
 Індеси: 11 12 13 14 15 16

1. Порівнюємо справа наліво: R=R, E=E, M=M, M=M, U=U, S=S.
2. **Шаблон знайдено.** Слово починається з індексу 11.

Аналіз складності

Ефективність у найкращому випадку

Найкращий випадок виникає тоді, коли кожне вирівнювання призводить до невідповідності на останньому символі шаблону, а символ у тексті, що викликав невідповідність, взагалі не з'являється в шаблоні. У цій ситуації відстань зсуву завжди становить m . Як наслідок, алгоритм перевіряє лише n/m символів тексту. Часова складність у найкращому випадку становить $\Omega(n/m)$, що значно краще, ніж продуктивність $\Omega(n)$ найкращого випадку алгоритму грубої сили.

Ефективність у найгіршому випадку

Найгірша поведінка спостерігається, коли значення зсувів постійно малі, а значна частина шаблону збігається з вікном тексту до виникнення невідповідності. Класичним таким прикладом є текст, що складається виключно з літер «А», і шаблон на зразок «ВАААА». У цьому випадку останній символ шаблону («А») завжди збігається з символом тексту. Невідповідність виникає лише на першому символі шаблону («В»). Коли шукається значення зсуву для символу «А» у тексті, алгоритм знаходить крайню праву «А» у перших $m-1$ позиціях шаблону, яка знаходиться в індексі $m-2$. Відстань зсуву таким чином становить $(m-1) - (m-2) = 1$. У такому випадку алгоритм поводить майже так само, як і метод грубої сили. Часова складність у найгіршому випадку становить $O(nm)$.

Висновок:

Алгоритм Хорспула є ефективним методом пошуку підрядка, який значно перевершує метод грубої сили завдяки використанню таблиці зсувів. Особливо добре алгоритм працює на великих текстах та при великому алфавіті. Попри те, що його найгірша складність залишається $O(nm)$, на практиці алгоритм демонструє високу ефективність.